Spring Boot Complete Guide

Comprehensive Interview Preparation & Hands-On Reference

Contents

Introduction **5**

1. Spring Boot Fundamentals **6**
- Explanation . . . 6
- What is Spring Boot? . . . 6
- Main Advantages . . . 6
- Auto-Configuration . . . 6
- Spring Boot Actuator . . . 7
- Hands-On Exercise . . . 7

2. Core Spring **8**
- Explanation . . . 8
- Dependency Injection . . . 8
- Stereotype Annotations . . . 8
- Spring Bean Lifecycle . . . 8
- Inversion of Control (IoC) . . . 9
- Constructor vs Field Injection . . . 9
- Hands-On Exercise . . . 9

3. Spring MVC **10**
- Explanation . . . 10
- @Controller vs @RestController . . . 10
- @RequestMapping & @PathVariable . . . 10
- @RequestBody . . . 10
- @ModelAttribute . . . 11
- Hands-On Exercise . . . 11

4. Spring Data JPA **12**
- Explanation . . . 12
- What is Spring Data JPA? . . . 12
- CrudRepository vs JpaRepository . . . 12
- Derived Query Methods . . . 12
- @Entity Annotation . . . 13
- @Id and @GeneratedValue . . . 13
- Hands-On Exercise . . . 13

5. Hibernate **15**
- Explanation . . . 15
- Hibernate and JPA Relationship . . . 15
- Lazy Loading vs Eager Loading . . . 15
- save() vs saveAndFlush() . . . 15

N+1 Query Problem . . . 15
Hands-On Exercise . . . 16

6. Spring Security **17**
Explanation . . . 17
What is Spring Security? . . . 17
Securing Endpoints . . . 17
Authentication vs Authorization . . . 18
UserDetailsService . . . 18
Roles and Authorities . . . 18
Hands-On Exercise . . . 18

7. REST APIs **19**
Explanation . . . 19
What is REST? . . . 19
HTTP Methods . . . 19
PUT vs PATCH . . . 19
ResponseEntity . . . 20
API Versioning . . . 20
Hands-On Exercise . . . 20

8. JWT Authentication **21**
Explanation . . . 21
What is JWT? . . . 21
JWT Structure . . . 21
JWT Implementation . . . 21
Session-based vs JWT Auth . . . 22
Hands-On Exercise . . . 22

9. Microservices **23**
Explanation . . . 23
What are Microservices? . . . 23
Service Discovery . . . 23
API Gateway . . . 23
Hands-On Exercise . . . 24

10. Exception Handling **25**
Explanation . . . 25
@ControllerAdvice . . . 25
Local vs Global Exception Handling . . . 25
Custom Exception Response . . . 26
Hands-On Exercise . . . 26

11. Transactions **27**
Explanation . . . 27
@Transactional Annotation . . . 27
Transaction Propagation . . . 27
Rollback in Spring Transactions . . . 27
Hands-On Exercise . . . 28

12. Caching **29**
Explanation . . . 29
Caching in Spring Boot . . . 29
@Cacheable . . . 29
@Cacheable, @CachePut, @CacheEvict . . . 30

Hands-On Exercise . . . 30

13. AOP & Proxy Mechanism **31**
Explanation . . . 31
What is AOP? . . . 31
AOP Setup . . . 31
Proxy Types . . . 32
Hands-On Exercise . . . 32

14. CAP Theorem **33**
Explanation . . . 33
What is CAP Theorem? . . . 33
System Types . . . 33
Choosing the Right System . . . 33
Hands-On Exercise . . . 33

15. Saga Pattern **35**
Explanation . . . 35
What is Saga Pattern? . . . 35
Saga Types . . . 35
Database Schema . . . 35
Implementation Example . . . 35
Hands-On Exercise . . . 36

16. CQRS & Event Sourcing **37**
Explanation . . . 37
CQRS (Command Query Responsibility Segregation) . . . 37
Event Sourcing . . . 37
Hands-On Exercise . . . 38

17. Outbox Pattern **39**
Explanation . . . 39
What is Outbox Pattern? . . . 39
Outbox Schema . . . 39
Implementation . . . 39
Hands-On Exercise . . . 40

18. Hibernate vs Spring Data JPA **41**
Explanation . . . 41
Architecture Layers . . . 41
Comparison Table . . . 41
Code Comparison . . . 41
Hands-On Exercise . . . 42

19. Native Query vs JPQL **43**
Explanation . . . 43
Comparison . . . 43
Examples . . . 43
Best Practices . . . 43
Hands-On Exercise . . . 44

20. Associations in JPA **45**
Explanation . . . 45
Association Types . . . 45
Examples . . . 45

Hands-On Exercise . . . 46

21. Inheritance in JPA **47**
Explanation . . . 47
Inheritance Strategies . . . 47
Comparison . . . 47
Hands-On Exercise . . . 48

22. Spring Boot Internal Working **49**
Explanation . . . 49
Startup Process . . . 49
ApplicationContext Refresh Process . . . 49
Bean Creation Process . . . 50
Hands-On Exercise . . . 50

23. ACID Properties **51**
Explanation . . . 51
What is ACID? . . . 51
Examples . . . 51
Hands-On Exercise . . . 51

24. Isolation Levels **52**
Explanation . . . 52
Isolation Levels . . . 52
Implementation . . . 52
Hands-On Exercise . . . 52

Quick Reference Card **54**
Spring Boot Starter Dependencies . . . 54
Common Annotations . . . 54
Suggested Study Path . . . 55

Introduction

This guide consolidates the core Spring Boot topics that come up most often in interviews, from framework fundamentals through distributed-systems patterns like Saga and CQRS. Every section follows the same structure:

1. **Concept explanation** — a plain-English description of *why* the feature exists, not just what it does.
2. **Code reference** — the original annotated snippets.
3. **Hands-On Exercise** — a short, self-contained task you can actually run to cement the concept.

Treat the exercises as checkpoints: if you can complete one in under fifteen minutes without looking anything up, you understand that topic well enough for an interview.

1. Spring Boot Fundamentals

Explanation

Plain Spring gave you full control but required heavy XML or Java configuration to wire up even a simple web app. Spring Boot's core idea is **convention over configuration**: it looks at what's on your classpath (e.g., a JDBC driver) and wires up sensible defaults automatically, while still letting you override anything. The embedded server means "run the jar" replaces "deploy the WAR to an app server," which is a big part of why Spring Boot fits microservices and containers so well.

What is Spring Boot?

Spring Boot is an opinionated framework that simplifies Spring application development by providing auto-configuration, embedded servers, and production-ready features.

```
@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

Main Advantages

- **Auto-Configuration**: Automatically configures beans based on dependencies
- **Standalone**: Embedded servers (Tomcat, Jetty, Undertow)
- **Production-Ready**: Actuator for health checks and metrics
- **Starter Dependencies**: Curated dependency sets
- **No XML**: Java-based configuration

Auto-Configuration

Auto-configuration automatically configures Spring beans based on:

1. Classpath dependencies
2. Configuration properties
3. Bean definitions

```
@Configuration
@ConditionalOnClass(DataSource.class)
@ConditionalOnProperty(prefix = "spring.datasource", name = "url")
public class DatabaseAutoConfiguration {
    @Bean
    public DataSource dataSource(DataSourceProperties properties) {
        return properties.initializeDataSourceBuilder().build();
    }
}
```

`@ConditionalOnClass` and `@ConditionalOnProperty` are the mechanism auto-configuration is built on — Spring only registers this bean if `DataSource` is on the classpath *and* a `spring.datasource.url` property exists. This is why adding `spring-boot-starter-data-jpa` "just works": the auto-configuration classes are already conditioned on the JPA classes that starter brings in.

Spring Boot Actuator

Provides production-ready features:

- `/actuator/health` — Application health
- `/actuator/info` — Application information
- `/actuator/metrics` — Application metrics
- `/actuator/env` — Environment properties

```
management.endpoints.web.exposure.include=health,info,metrics,env
```

Hands-On Exercise

1. Create a new project at start.spring.io with `Spring Web` and `Spring Boot Actuator`.
2. Run it and hit `http://localhost:8080/actuator/health` — confirm you get `{"status":"UP"}`.
3. Add `management.endpoint.health.show-details=always` to `application.properties`, restart, and observe the extra diagnostic detail (disk space, ping) that appears.
4. Write a custom `@Component implements HealthIndicator` that reports `DOWN` if a fake external dependency flag is false, and confirm it shows up under `/actuator/health`.

2. Core Spring

Explanation

Dependency Injection (DI) and Inversion of Control (IoC) are the foundation everything else in Spring sits on. Instead of a class calling `new PaymentService()` itself (tight coupling, hard to test), the container builds the object graph and *hands* each class what it needs. This is what makes mocking dependencies in unit tests trivial, and it's why constructor injection is preferred: it makes "this class cannot exist without a `PaymentService`" explicit and enforced by the compiler.

Dependency Injection

Dependency Injection is a design pattern where objects receive their dependencies from an external source.

```
// Constructor Injection (Recommended)
@Service
public class OrderService {
    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}

// Field Injection (Not Recommended)
@RestController
public class UserController {
    @Autowired
    private UserService userService;
}
```

Stereotype Annotations

Annotation	Layer	Purpose
`@Component`	Generic	Any Spring bean
`@Service`	Business	Business logic
`@Repository`	Data	Data access
`@Controller`	Web	MVC controller
`@RestController`	REST	REST API

Spring Bean Lifecycle

```
Bean Definition → Instantiation → Properties Set →
BeanName/Context Aware → Initialization → Ready → Destruction
```

```
@Component
public class LifecycleBean implements InitializingBean, DisposableBean {
    @PostConstruct
    public void init() { /* After properties set */ }

    @Override
```

```
    public void afterPropertiesSet() { /* After properties set */ }

    @PreDestroy
    public void destroy() { /* Before destruction */ }
}
```

Inversion of Control (IoC)

IoC is a principle where control of object creation and lifecycle is inverted from application code to a container.

Constructor vs Field Injection

Aspect	Constructor	Field Injection
Immutability	Yes: Fields can be final	No: Fields mutable
Testability	Yes: Easy to mock	No: Requires reflection
Null Safety	Yes: Required dependencies	No: May be null

Hands-On Exercise

1. Create `PaymentService` as an interface with two implementations, `CreditCardPaymentService` and `PaypalPaymentService`, both annotated `@Service` with distinct bean names via `@Qualifier`.
2. Inject the desired implementation into `OrderService` using constructor injection and `@Qualifier`.
3. Write a JUnit test for `OrderService` that passes a **mock** `PaymentService` directly to the constructor — no Spring context needed. This demonstrates why constructor injection makes testing easy.
4. Add a `@PostConstruct` method to `OrderService` that logs “OrderService ready” and confirm it prints once at startup.

3. Spring MVC

Explanation

Spring MVC is the layer that turns HTTP requests into Java method calls and Java return values back into HTTP responses. The key distinction to internalize is `@Controller` (returns a *view name* to be rendered, e.g., Thymeleaf/JSP) versus `@RestController` (writes the return value directly to the response body as JSON/XML via `HttpMessageConverters`). Almost all modern API work uses `@RestController`.

@Controller vs @RestController

```java
// @Controller - Returns view
@Controller
public class ViewController {
    @GetMapping("/home")
    public String home(Model model) {
        model.addAttribute("message", "Hello");
        return "home";  // View name
    }
}

// @RestController - Returns JSON
@RestController
@RequestMapping("/api/users")
public class UserRestController {
    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id) {
        return userService.findById(id);  // JSON response
    }
}
```

@RequestMapping & @PathVariable

```java
@RestController
@RequestMapping("/api/orders")
public class OrderController {
    @GetMapping("/{orderId}")
    public Order getOrder(@PathVariable Long orderId) {
        return orderService.findById(orderId);
    }
}
```

@RequestBody

Binds HTTP request body to a Java object using `HttpMessageConverter`.

```java
@PostMapping("/users")
public User createUser(@RequestBody User user) {
    return userService.save(user);
}
```

@ModelAttribute

Binds request parameters to model objects.

```
@Controller
public class UserController {
    @ModelAttribute("countries")
    public List<String> getCountries() {
        return Arrays.asList("USA", "UK", "India");
    }

    @PostMapping("/register")
    public String register(@ModelAttribute User user) {
        return "success";
    }
}
```

Hands-On Exercise

1. Build a `@RestController` with a `GET /api/books/{id}` endpoint and a `GET /api/books?author=X` endpoint using `@RequestParam`.
2. Add a `POST /api/books` endpoint that accepts a JSON body via `@RequestBody` and returns `201 Created`.
3. Use `curl` or Postman to confirm all three endpoints work, and inspect the `Content-Type` header of the response.
4. Convert one endpoint to a traditional `@Controller` returning a Thymeleaf view, to feel the difference between “returning data” and “returning a view name.”

4. Spring Data JPA

Explanation

Spring Data JPA removes almost all boilerplate DAO code. You declare an interface extending `JpaRepository<Entity, IdType>` and Spring generates the implementation at runtime — both the standard CRUD methods and "derived queries" it parses out of your method names (e.g., `findByLastNameAndAgeGreaterThan`). For anything the naming convention can't express, you drop to `@Query` with JPQL or native SQL.

What is Spring Data JPA?

Spring Data JPA simplifies data access by providing repository interfaces with CRUD operations, derived queries, and custom query methods.

```
@Repository
public interface UserRepository extends JpaRepository<User, Long> {
    // Derived queries
    List<User> findByEmailContaining(String email);

    // Custom JPQL
    @Query("SELECT u FROM User u WHERE u.age > :minAge")
    List<User> findAdults(@Param("minAge") int minAge);
}
```

CrudRepository vs JpaRepository

Repository	Methods	Use Case
`CrudRepository`	`save()`, `findById()`, `delete()`	Simple CRUD
`JpaRepository`	`flush()`, `saveAndFlush()`, `getOne()`	JPA-specific needs

Derived Query Methods

```
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    // Simple equality
    List<Employee> findByFirstName(String firstName);

    // Multiple properties
    List<Employee> findByFirstNameAndLastName(String firstName, String lastName);

    // Comparison
    List<Employee> findBySalaryGreaterThan(BigDecimal salary);

    // String operations
    List<Employee> findByFirstNameContaining(String partial);

    // Sorting
    List<Employee> findByDepartmentOrderByLastNameAsc(String dept);
}
```

@Entity Annotation

```java
@Entity
@Table(name = "products")
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "product_name", nullable = false, length = 100)
    private String name;

    @Column(precision = 10, scale = 2)
    private BigDecimal price;

    @Temporal(TemporalType.TIMESTAMP)
    private Date createdDate;

    @Enumerated(EnumType.STRING)
    private ProductStatus status;

    @Transient  // Not persisted
    private String temporaryField;

    @Version  // Optimistic locking
    private Long version;
}
```

@Id and @GeneratedValue

```java
@Entity
public class Product {
    // 1. IDENTITY - Auto-increment
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // 2. SEQUENCE - Database sequence
    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "product_seq")
    @SequenceGenerator(name = "product_seq", sequenceName = "product_sequence")
    private Long id;

    // 3. UUID - Generate UUID
    @Id
    @GeneratedValue(generator = "uuid2")
    @GenericGenerator(name = "uuid2", strategy = "uuid2")
    private String id;
}
```

Hands-On Exercise

1. Spin up an H2 in-memory database and create a `Product` entity with `id`, `name`, `price`, `status`.

2. Write a `ProductRepository` with three derived queries: by name containing a substring, by price greater than a value, and sorted by name descending.
3. Add one `@Query` method using JPQL that joins nothing but filters on an enum status.
4. Write a small `CommandLineRunner` that saves five products and prints the results of each query, so you can see the generated SQL in the console (`spring.jpa.show-sql=true`).

5. Hibernate

Explanation

Hibernate is the default JPA provider Spring Boot wires in. The trickiest real-world Hibernate topic is **lazy vs. eager loading** combined with the **N+1 query problem**: if you loop over a list of parent entities and touch a lazily-loaded collection on each one, Hibernate silently fires one query per parent. `JOIN FETCH` or `@EntityGraph` collapses that into a single query.

Hibernate and JPA Relationship

- JPA is a specification
- Hibernate is an implementation of JPA

```
<!-- Spring Boot Starter includes both -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

Lazy Loading vs Eager Loading

Fetch Type	When Loaded	Use Case
`LAZY`	On first access	Collections, large data
`EAGER`	Immediately with parent	Small, always-needed data

```
@Entity
public class Order {
    @OneToMany(fetch = FetchType.LAZY)
    private List<OrderItem> items;  // Load only when accessed

    @ManyToOne(fetch = FetchType.EAGER)
    private Customer customer;  // Load immediately
}
```

save() vs saveAndFlush()

Method	Behavior	Use Case
`save()`	Delayed flush	Bulk inserts
`saveAndFlush()`	Immediate flush	When ID needed immediately

N+1 Query Problem

Performance issue where one query causes additional queries for each association.

```
// Problem
List<Order> orders = orderRepository.findAll();
for (Order order : orders) {
    order.getItems();  // N additional queries!
}
```

```java
// Solution 1: JOIN FETCH
@Query("SELECT DISTINCT o FROM Order o JOIN FETCH o.items")
List<Order> findAllWithItems();

// Solution 2: Entity Graph
@NamedEntityGraph(name = "Order.withItems",
    attributeNodes = @NamedAttributeNode("items"))
@Entity
public class Order { /*...*/ }

@Repository
public interface OrderRepository {
    @EntityGraph("Order.withItems")
    List<Order> findAll();
}
```

Hands-On Exercise

1. Create `Order` and `OrderItem` entities with a lazy `@OneToMany`.
2. Load all orders and iterate, calling `.getItems().size()` on each — turn on `spring.jpa.show-sql=true` and count the queries printed (you should see N+1).
3. Add a `JOIN FETCH` query and re-run — confirm it now takes exactly one query.
4. Repeat using `@EntityGraph` instead of `JOIN FETCH`, and compare the generated SQL between the two approaches.

6. Spring Security

Explanation

Spring Security answers two separate questions: **authentication** ("who are you?") and **authorization** ("what are you allowed to do?"). The `SecurityFilterChain` is a chain-of-responsibility pipeline every request passes through before reaching your controller; `UserDetailsService` is the plug-in point where you tell Spring how to load a user's credentials and authorities from your own data store.

What is Spring Security?

Spring Security is a comprehensive security framework providing authentication, authorization, and protection against common attacks.

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Securing Endpoints

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(authz -> authz
                .requestMatchers("/public/**").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .requestMatchers("/user/**").hasAnyRole("USER", "ADMIN")
                .anyRequest().authenticated()
            )
            .formLogin(form -> form
                .loginPage("/login")
                .defaultSuccessUrl("/dashboard")
            );
        return http.build();
    }
}

@RestController
public class SecuredController {
    @PreAuthorize("hasRole('ADMIN')")
    @GetMapping("/admin/users")
    public List<User> getUsers() {
        return userService.findAll();
    }

    @PreAuthorize("hasAuthority('USER_READ')")
    @GetMapping("/users/{id}")
    public User getUser(@PathVariable Long id) {
        return userService.findById(id);
```

```
    }
}
```

Authentication vs Authorization

Aspect	Authentication	Authorization
Purpose	Verify identity	Verify permissions
Question	“Who are you?”	“What can you do?”
Process	Login, credentials	Access control

UserDetailsService

```
@Service
public class CustomUserDetailsService implements UserDetailsService {
    @Override
    public UserDetails loadUserByUsername(String username) {
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("User not found"));

        return new org.springframework.security.core.userdetails.User(
            user.getUsername(),
            user.getPassword(),
            getAuthorities(user)
        );
    }
}
```

Roles and Authorities

```
// Roles - Group of authorities
public enum Role {
    USER(Set.of(Permission.USER_READ, Permission.USER_WRITE)),
    ADMIN(Set.of(Permission.USER_READ, Permission.USER_WRITE, Permission.USER_DELETE));

    private final Set<Permission> permissions;
}

// Usage
@PreAuthorize("hasRole('ADMIN')")  // Role-based
@PreAuthorize("hasAuthority('USER_WRITE')")  // Authority-based
```

Hands-On Exercise

1. Add `spring-boot-starter-security` to a project and confirm every endpoint now demands a login (the default generated password appears in the console).
2. Implement `CustomUserDetailsService` backed by an in-memory `Map<String, User>` with two users: one `USER` role, one `ADMIN` role.
3. Configure a `SecurityFilterChain` that permits `/public/**`, requires `ADMIN` for `/admin/**`, and requires authentication for everything else.
4. Add `@PreAuthorize("hasRole('ADMIN')")` on one controller method and confirm a `USER`-role login gets `403 Forbidden`.

7. REST APIs

Explanation

REST is a set of architectural constraints, not a protocol — the interview-relevant part is knowing which HTTP methods are meant to be idempotent, and how `ResponseEntity` gives you full control over status codes and headers instead of Spring guessing for you.

What is REST?

```java
@RestController
@RequestMapping("/api/products")
public class ProductRestController {
    @GetMapping
    public ResponseEntity<List<Product>> getProducts() {
        return ResponseEntity.ok(productService.findAll());
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Product createProduct(@RequestBody Product product) {
        return productService.save(product);
    }
}
```

HTTP Methods

Method	Purpose	Idempotent
GET	Read data	Yes: Yes
POST	Create resource	No: No
PUT	Replace entire resource	Yes: Yes
PATCH	Partial update	Yes: Yes
DELETE	Remove resource	Yes: Yes

PUT vs PATCH

```java
// PUT - Complete replacement
@PutMapping("/{id}")
public User updateUser(@PathVariable Long id, @RequestBody User user) {
    // Replaces ALL fields
    return userService.replace(id, user);
}

// PATCH - Partial update
@PatchMapping("/{id}")
public User patchUser(@PathVariable Long id, @RequestBody JsonPatch patch) {
    // Updates ONLY provided fields
    return userService.patch(id, patch);
}
```

ResponseEntity

```java
@GetMapping("/{id}")
public ResponseEntity<User> getUser(@PathVariable Long id) {
    return userService.findById(id)
        .map(ResponseEntity::ok)
        .orElse(ResponseEntity.notFound().build());
}

@PostMapping
public ResponseEntity<User> createUser(@Valid @RequestBody User user) {
    User saved = userService.save(user);
    URI location = ServletUriComponentsBuilder
        .fromCurrentRequest()
        .path("/{id}")
        .buildAndExpand(saved.getId())
        .toUri();
    return ResponseEntity.created(location).body(saved);
}
```

API Versioning

```java
// 1. URI Versioning (Most Common)
@RequestMapping("/api/v1/products")
public class ProductControllerV1 { /*...*/ }

@RequestMapping("/api/v2/products")
public class ProductControllerV2 { /*...*/ }

// 2. Header Versioning
@GetMapping(headers = "API-Version=1")
public List<ProductV1DTO> getProductsV1() { /*...*/ }

// 3. Media Type Versioning
@GetMapping(produces = "application/vnd.company.app-v1+json")
public List<ProductV1DTO> getProductsV1() { /*...*/ }
```

Hands-On Exercise

1. Build `GET`, `POST`, `PUT`, `PATCH`, `DELETE` endpoints for a `Task` resource.
2. Use `ResponseEntity` everywhere: return `404` for missing tasks, `201` with a `Location` header for creation, `204` for delete.
3. Implement `PATCH` using a simple partial-DTO approach (only non-null fields overwrite existing ones) rather than full JSON Patch, and confirm updating one field leaves the others untouched.
4. Add a second controller under `/api/v2/tasks` that returns a different DTO shape, to practice URI-based versioning.

8. JWT Authentication

Explanation

JWTs let you do stateless authentication: once issued, the token itself carries the claims (subject, roles, expiry) and is verified with a signature check — no server-side session lookup needed. This is what makes JWTs scale horizontally so easily, at the cost of harder revocation (you can't "log out" a JWT server-side without extra infrastructure like a blocklist).

What is JWT?

JWT (JSON Web Token) is a compact, URL-safe token format for securely transmitting information.

JWT Structure

```
Header.Payload.Signature
```

JWT Implementation

```
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.11.5</version>
</dependency>
```

```
@Component
public class JwtTokenUtil {
    @Value("${jwt.secret}")
    private String secret;

    private final Key key = Keys.hmacShaKeyFor(secret.getBytes());

    public String generateToken(UserDetails userDetails) {
        return Jwts.builder()
            .setSubject(userDetails.getUsername())
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + expiration))
            .claim("authorities", userDetails.getAuthorities())
            .signWith(key, SignatureAlgorithm.HS512)
            .compact();
    }

    public boolean validateToken(String token) {
        try {
            Jwts.parserBuilder().setSigningKey(key).build().parseClaimsJws(token);
            return true;
        } catch (JwtException e) {
            return false;
        }
    }
}
```

Session-based vs JWT Auth

Aspect	Session-based	JWT-based
State	Stateful	Stateless
Storage	Server memory/database	Client stores token
Scalability	Requires session replication	Naturally scalable
Security	CSRF protection needed	No CSRF

Hands-On Exercise

1. Add `jjwt-api`, `jjwt-impl`, and `jjwt-jackson` to a project.
2. Implement `JwtTokenUtil.generateToken()` and `validateToken()`, and write a small `main` method or test that generates a token, prints it, and validates it.
3. Add a `JwtAuthenticationFilter extends OncePerRequestFilter` that reads the `Authorization: Bearer <token>` header and sets the `SecurityContext` if valid.
4. Wire the filter into your `SecurityFilterChain` before `UsernamePasswordAuthenticationFilter`, then confirm a protected endpoint rejects requests with no token and accepts ones with a valid token.

9. Microservices

Explanation

The core idea of microservices is independent deployability: each service owns its data and can be released on its own schedule. That independence introduces new problems plain monoliths don't have — services need to *find* each other (service discovery) and clients need a single, stable entry point instead of talking to a dozen internal hostnames directly (API gateway).

What are Microservices?

```
@SpringBootApplication
@EnableDiscoveryClient
@RestController
@RequestMapping("/api/products")
public class ProductServiceApplication {
    @GetMapping
    public List<Product> getProducts() {
        return productRepository.findAll();
    }
}
```

Service Discovery

```
@SpringBootApplication
@EnableEurekaServer
public class ServiceRegistryApplication { /*...*/ }

@SpringBootApplication
@EnableDiscoveryClient
public class UserServiceApplication { /*...*/ }

@Service
public class ServiceClient {
    @Autowired
    @LoadBalanced
    private RestTemplate restTemplate;

    public Product getProduct(Long id) {
        return restTemplate.getForObject(
            "http://product-service/api/products/" + id,
            Product.class
        );
    }
}
```

API Gateway

```
@Configuration
public class GatewayConfig {
    @Bean
    public RouteLocator customRouteLocator(RouteLocatorBuilder builder) {
```

```
        return builder.routes()
            .route("user-service", r -> r
                .path("/api/users/**")
                .filters(f -> f
                    .addRequestHeader("X-Request-Id", UUID.randomUUID().toString())
                    .circuitBreaker(config -> config
                        .setName("userServiceCB")
                        .setFallbackUri("forward:/fallback/users"))
                )
                .uri("lb://USER-SERVICE"))
            .build();
    }
}
```

Hands-On Exercise

1. Create three Spring Boot apps: a Eureka server, a `product-service`, and an `order-service` that calls `product-service` via a `@LoadBalanced RestTemplate`.
2. Register both services with Eureka and confirm they appear on the Eureka dashboard.
3. Stop `product-service` and observe what happens to `order-service` calls — then add a simple fallback (return a cached/default product) using a circuit breaker.
4. Add a minimal Spring Cloud Gateway app in front of both services and route `/api/products/**` and `/api/orders/**` to the right service.

10. Exception Handling

Explanation

Without centralized handling, every controller ends up littered with try/catch blocks that duplicate error-formatting logic. `@RestControllerAdvice` intercepts exceptions thrown anywhere in the request-handling flow and lets you map each exception type to a consistent, well-shaped error response — critical for APIs consumed by other teams or external clients.

@ControllerAdvice

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(BusinessException.class)
    public ResponseEntity<ErrorResponse> handleBusinessException(BusinessException ex) {
        ErrorResponse error = ErrorResponse.builder()
            .timestamp(LocalDateTime.now())
            .status(ex.getStatus().value())
            .message(ex.getMessage())
            .build();
        return new ResponseEntity<>(error, ex.getStatus());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidationExceptions(
            MethodArgumentNotValidException ex) {
        Map<String, String> errors = ex.getBindingResult()
            .getFieldErrors()
            .stream()
            .collect(Collectors.toMap(
                FieldError::getField,
                FieldError::getDefaultMessage
            ));

        return ResponseEntity.badRequest()
            .body(new ErrorResponse("Validation failed", errors.toString()));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGenericException(Exception ex) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(new ErrorResponse("Internal server error", ex.getMessage()));
    }
}
```

Local vs Global Exception Handling

Aspect	Local `@ExceptionHandler`	Global `@ControllerAdvice`
Scope	Single controller	All controllers
Priority	Higher priority	Lower priority
Use Case	Controller-specific logic	Application-wide handling

Custom Exception Response

```
@Data
@Builder
public class ErrorResponse {
    private String traceId;
    private LocalDateTime timestamp;
    private int status;
    private String error;
    private String code;
    private String message;
    private List<String> details;
}

public class CustomBusinessException extends RuntimeException {
    private final String errorCode;
    private final HttpStatus status;

    public CustomBusinessException(String errorCode, String message, HttpStatus status) {
        super(message);
        this.errorCode = errorCode;
        this.status = status;
    }
}
```

Hands-On Exercise

1. Create a `ResourceNotFoundException` and throw it from a service when an entity id doesn't exist.
2. Build a `@RestControllerAdvice` that maps it to `404` with a JSON body containing `timestamp`, `status`, and `message`.
3. Add a handler for `MethodArgumentNotValidException` (trigger it with a `@Valid @RequestBody` DTO that has a `@NotBlank` field) and confirm the response lists which field failed.
4. Add a catch-all `Exception.class` handler last, and confirm handler *order* — throw a `ResourceNotFoundException` and verify the specific handler fires, not the generic one.

11. Transactions

Explanation

`@Transactional` wraps a method in a database transaction so that either all its writes commit or none do. The subtlety interviewers probe is **propagation**: what happens when a `@Transactional` method calls another `@Transactional` method — does it join the existing transaction (`REQUIRED`, the default) or start an independent one (`REQUIRES_NEW`)? That choice determines whether a failure in the inner call rolls back the outer one too.

@Transactional Annotation

```java
@Service
@Transactional
public class OrderService {
    @Transactional
    public Order createOrder(Order order) {
        // All operations in one transaction
        Order savedOrder = orderRepository.save(order);
        inventoryService.reserveStock(savedOrder.getItems());
        paymentService.processPayment(savedOrder);
        return savedOrder;
    }

    @Transactional(readOnly = true)
    public List<Order> getTodayOrders() {
        return orderRepository.findByDate(LocalDate.now());
    }
}
```

Transaction Propagation

Propagation	Behavior	Use Case
`REQUIRED`	Use existing, create new if none	Most operations
`REQUIRES_NEW`	Always create new	Logging, auditing
`NESTED`	Create nested transaction	Partial rollback
`SUPPORTS`	Use existing if available	Read operations
`MANDATORY`	Must have transaction	Critical operations
`NEVER`	Must not have transaction	Non-transactional ops

Rollback in Spring Transactions

```java
// Default: rollback on RuntimeException
@Transactional
public void defaultRollback() {
    throw new RuntimeException("Rollback all changes");
}

// Custom rollback rules
@Transactional(rollbackFor = {BusinessException.class})
public void customRollback() throws BusinessException {
    throw new BusinessException("Rollback this");
```

```
}

// No rollback for specific exceptions
@Transactional(noRollbackFor = {BusinessException.class})
public void noRollbackForException() throws BusinessException {
    throw new BusinessException("No rollback");
}

// Manual rollback
@Transactional
public void manualRollback() {
    try {
        // Operations
    } catch (Exception e) {
        TransactionAspectSupport.currentTransactionStatus().setRollbackOnly();
    }
}
```

Hands-On Exercise

1. Write a `@Transactional` method that saves two entities and then throws a `RuntimeException` — confirm neither entity is persisted.
2. Change the thrown exception to a checked exception and confirm the default behavior does **not** roll back; then fix it with `rollbackFor`.
3. Create two service methods, `A()` (propagation `REQUIRED`) calling `B()` (propagation `REQUIRES_NEW`). Throw an exception in `A()` after `B()` returns, and confirm `B()`'s work survives while `A()`'s does not.
4. Add `@Transactional(readOnly = true)` to a query-only method and use `spring.jpa.show-sql=true` to observe there's no flush/commit overhead compared to a writing transaction.

12. Caching

Explanation

Caching trades a small amount of staleness risk for a large reduction in database load. Spring's caching abstraction is declarative: `@Cacheable` short-circuits the method body if the key is already cached, `@CachePut` always runs the method but refreshes the cache with the result, and `@CacheEvict` removes stale entries — typically called from the method that changes the underlying data.

Caching in Spring Boot

```
@Configuration
@EnableCaching
public class CacheConfig {
    @Bean
    public CacheManager cacheManager() {
        return new ConcurrentMapCacheManager("users", "products");
    }
}

@Configuration
@EnableCaching
public class RedisCacheConfig {
    @Bean
    public RedisCacheManager cacheManager(RedisConnectionFactory connectionFactory) {
        RedisCacheConfiguration config = RedisCacheConfiguration.defaultCacheConfig()
            .entryTtl(Duration.ofMinutes(10))
            .disableCachingNullValues();

        return RedisCacheManager.builder(connectionFactory)
            .cacheDefaults(config)
            .build();
    }
}
```

@Cacheable

```
@Service
public class UserService {
    @Cacheable(value = "users", key = "#id")
    public User getUserById(Long id) {
        System.out.println("Fetching from database: " + id);
        return userRepository.findById(id).orElseThrow();
    }

    @Cacheable(value = "users", condition = "#id > 1000")
    public User getImportantUser(Long id) {
        // Only cache if ID > 1000
        return userRepository.findById(id).orElseThrow();
    }
}
```

@Cacheable, @CachePut, @CacheEvict

Annotation	Purpose	Behavior
`@Cacheable`	Get from cache or execute	Cache if not present
`@CachePut`	Update cache	Always execute and update cache
`@CacheEvict`	Remove from cache	Remove entry/entries from cache

```java
@Service
public class ProductService {
    @Cacheable(value = "products", key = "#id")
    public Product getProduct(Long id) { /*...*/ }

    @CachePut(value = "products", key = "#product.id")
    public Product updateProduct(Product product) { /*...*/ }

    @CacheEvict(value = "products", key = "#id")
    public void deleteProduct(Long id) { /*...*/ }

    @CacheEvict(value = "products", allEntries = true)
    public void clearAllProducts() { /*...*/ }
}
```

Hands-On Exercise

1. Add `@EnableCaching` with `ConcurrentMapCacheManager` and put `@Cacheable` on a "slow" service method (add a `Thread.sleep(2000)` to simulate a slow DB call).
2. Call it twice with the same id and confirm the second call returns instantly and the "Fetching from database" log line only prints once.
3. Add an `updateProduct` method with `@CachePut` and confirm calling it updates the cached value without a separate read.
4. Add a `@CacheEvict(allEntries = true)` admin endpoint and confirm it clears the cache — the next `getProduct` call is slow again.

13. AOP & Proxy Mechanism

Explanation

AOP lets you factor out concerns that cut across many unrelated classes — logging, timing, security checks — into a single module instead of repeating them in every method. Under the hood Spring implements this with **proxies**: a JDK dynamic proxy (if the bean implements an interface) or a CGLIB subclass proxy (if it doesn't), which is why AOP advice silently fails to trigger on private methods or on self-invocation (a method calling another method on `this` bypasses the proxy entirely).

What is AOP?

```
/*
    AOP Core Concepts:
    Aspect     → Module encapsulating cross-cutting concern
    Join Point → Point during program execution
    Advice     → Action taken at a join point
    Pointcut   → Expression matching join points
    Proxy      → Object intercepting method calls
*/
```

AOP Setup

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

```
@Configuration
@EnableAspectJAutoProxy
public class AopConfig { /*...*/ }

@Aspect
@Component
public class LoggingAspect {
    @Pointcut("execution(* com.example.service.*.*(..))")
    public void serviceLayer() {}

    @Before("serviceLayer()")
    public void logBefore(JoinPoint joinPoint) {
        System.out.println("Before: " + joinPoint.getSignature().getName());
    }

    @AfterReturning(value = "serviceLayer()", returning = "result")
    public void logAfterReturning(JoinPoint joinPoint, Object result) {
        System.out.println("Returned: " + result);
    }

    @Around("serviceLayer()")
    public Object logAround(ProceedingJoinPoint joinPoint) throws Throwable {
        long start = System.currentTimeMillis();
        Object result = joinPoint.proceed();
        long elapsed = System.currentTimeMillis() - start;
```

```
        System.out.println("Time: " + elapsed + "ms");
        return result;
    }
}
```

Proxy Types

Feature	JDK Proxy	CGLIB Proxy
Requirement	Interface	No interface needed
Performance	Better	Slightly slower
Limitations	Must have interface	Final methods/classes
Default	If interface exists	If no interface

Hands-On Exercise

1. Write a `@Pointcut` matching every method in your service package and add `@Before` / `@AfterReturning` advice that logs entry and exit.
2. Add an `@Around` advice that times execution and confirm it prints milliseconds for a method with an artificial `Thread.sleep`.
3. Call one service method from *within* another method on the same class (self-invocation) and confirm the aspect does **not** trigger — then fix it by injecting the service into itself (or refactor into a separate bean) and confirm the aspect now fires.
4. Make your service class implement an interface and observe (via debugger or logging the object's class name) that Spring switches from a CGLIB subclass proxy to a JDK dynamic proxy.

14. CAP Theorem

Explanation

CAP theorem is about *trade-offs under network partition*, not a static property of a system in general. When a partition happens, you can either keep serving requests with possibly stale data (AP) or refuse requests until consistency is restored (CP) — you cannot do both. In practice most systems aren't purely one or the other; they choose per-operation (e.g., strongly consistent writes, eventually-consistent reads).

What is CAP Theorem?

In a distributed data store, it's impossible to simultaneously guarantee all three properties:

```
CAP Theorem:
- Consistency (C): All nodes see the same data
- Availability (A): Every request gets a response
- Partition Tolerance (P): System continues despite network failures

You can only have 2 out of 3:
- CA: Consistency + Availability (no Partition Tolerance)
- CP: Consistency + Partition Tolerance (no Availability)
- AP: Availability + Partition Tolerance (no Consistency)
```

System Types

System Type	Consistency	Availability	Partition Tolerance	Use Cases
CP	Yes: Strong	No: May fail	Yes: Tolerant	Banking, Financial
AP	No: Eventual	Yes: High	Yes: Tolerant	Social Media
CA	Yes: Strong	Yes: High	No: Not tolerant	Single DB

Choosing the Right System

```java
public class CAPDecision {
    public String chooseSystem(Requirements req) {
        if (req.isFinancialTransaction()) {
            return "CP System";
        } else if (req.isSocialMedia()) {
            return "AP System";
        } else {
            return "CA System";
        }
    }
}
```

Hands-On Exercise

1. On paper (or in a short write-up), classify these systems as CP or AP and justify why: a bank ledger, a "likes" counter on a social post, a distributed lock service, a product-catalog cache.

2. Spin up a 3-node etcd or ZooKeeper cluster locally (or read their docs) and identify which CAP category they target.
3. Read about DynamoDB's tunable consistency (`eventually consistent` vs `strongly consistent` reads) and explain in your own words how it lets a single system offer both AP and CP behavior depending on the request.

15. Saga Pattern

Explanation

In a monolith, a single database transaction can span multiple tables. In microservices, each service has its own database, so you can't use a single ACID transaction across an order, payment, and inventory service. The Saga pattern replaces that with a sequence of local transactions, each with a matching **compensating transaction** to undo it if a later step fails.

What is Saga Pattern?

The Saga pattern is a sequence of local transactions where each transaction updates data within a single service.

Saga Types

Type	Description	Pros	Cons
Choreography	Decentralized, event-driven	Loosely coupled	Complex logic distribution
Orchestration	Centralized coordinator	Centralized logic	Single point of failure

Database Schema

```sql
-- Core Tables
CREATE TABLE saga_instances (
    saga_id UUID PRIMARY KEY,
    saga_type VARCHAR(100) NOT NULL,
    status VARCHAR(50) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE saga_steps (
    step_id UUID PRIMARY KEY,
    saga_id UUID REFERENCES saga_instances(saga_id),
    step_order INTEGER NOT NULL,
    step_name VARCHAR(100) NOT NULL,
    status VARCHAR(50) NOT NULL
);

CREATE TABLE saga_compensations (
    compensation_id UUID PRIMARY KEY,
    saga_id UUID REFERENCES saga_instances(saga_id),
    status VARCHAR(50) NOT NULL
);
```

Implementation Example

```
@Transactional
public void executeSaga() {
```

```
    try {
        // Step 1: Create Order
        Order order = orderService.createOrder(request);

        // Step 2: Process Payment
        Payment payment = paymentService.processPayment(order);

        // Step 3: Reserve Inventory
        inventoryService.reserveStock(order.getItems());

    } catch (Exception e) {
        // Compensate in reverse order
        inventoryService.releaseStock();
        paymentService.refundPayment();
        orderService.cancelOrder();
    }
}
```

Hands-On Exercise

1. Model a three-step saga (create order → reserve inventory → charge payment) as three separate in-memory services (no real DB needed).
2. Implement the orchestration version: a `SagaOrchestrator` calls each step in order and, on any failure, calls compensating methods in reverse order.
3. Force the "charge payment" step to throw on purpose and confirm the orchestrator releases the inventory and cancels the order.
4. As a stretch goal, sketch (in a short doc, no code required) how the same saga would look under the choreography style, with each service publishing an event the next service listens for.

16. CQRS & Event Sourcing

Explanation

CQRS separates the model used to *write* data (commands) from the model used to *read* it (queries) — useful when read and write workloads have very different shapes or scaling needs. Event Sourcing takes this further: instead of storing current state, you store the sequence of events that led to it, and derive current state by replaying them. Together they're common in systems needing a full audit trail or complex read projections.

CQRS (Command Query Responsibility Segregation)

```
// Command Side (Write)
@RestController
@RequestMapping("/api/orders")
public class CommandController {
    @PostMapping
    public OrderCommandResult createOrder(@RequestBody CreateOrderCommand command) {
        return commandHandler.handle(command);
    }
}

// Query Side (Read)
@RestController
@RequestMapping("/api/orders")
public class QueryController {
    @GetMapping("/{orderId}")
    public OrderQueryResult getOrder(@PathVariable String orderId) {
        return queryHandler.getOrder(orderId);
    }
}
```

Event Sourcing

```
// Domain Events
public class OrderCreatedEvent extends DomainEvent {
    private String customerId;
    private BigDecimal amount;
}

// Event Store
@Repository
public class EventStore {
    @Transactional
    public void saveEvents(String aggregateId, List<DomainEvent> events) {
        for (DomainEvent event : events) {
            jdbcTemplate.update(
                "INSERT INTO event_store (aggregate_id, event_type, event_data, version) " +
                "VALUES (?, ?, ?::jsonb, ?)",
                aggregateId, event.getEventType(), toJson(event), event.getVersion()
            );
        }
    }
}
```

```java
// Aggregate
@Getter
public class OrderAggregate {
    private String orderId;
    private OrderStatus status;
    private int version;

    public void applyEvent(DomainEvent event) {
        if (event instanceof OrderCreatedEvent) {
            applyOrderCreated((OrderCreatedEvent) event);
        } else if (event instanceof OrderPaidEvent) {
            applyOrderPaid((OrderPaidEvent) event);
        }
        version++;
    }
}
```

Hands-On Exercise

1. Model an `OrderAggregate` whose state is entirely derived from a list of events (`OrderCreated`, `OrderPaid`, `OrderShipped`).
2. Write an `EventStore` (even a simple `List<DomainEvent>` in memory) and a function `replay(List<DomainEvent>) -> OrderAggregate`.
3. Confirm that replaying the same event list twice always produces an identical aggregate state (determinism is the point of event sourcing).
4. Add a separate read-side "projection" that listens to the same events and maintains a flattened `OrderSummary` table optimized for a dashboard query — this is the "Q" side of CQRS.

17. Outbox Pattern

Explanation

If a service saves data to its database *and* publishes a message to Kafka in two separate steps, a crash between those steps causes a data/message mismatch. The Outbox pattern writes both the business change and an "event to publish" row in the **same local database transaction**, then a separate poller reads unpublished rows and sends them — guaranteeing "at least once" delivery without needing distributed transactions.

What is Outbox Pattern?

The Outbox Pattern ensures reliable message delivery by storing events in the database transactionally and publishing them asynchronously.

Outbox Schema

```sql
CREATE TABLE outbox_messages (
    id BIGSERIAL PRIMARY KEY,
    event_id VARCHAR(36) NOT NULL UNIQUE,
    aggregate_type VARCHAR(50) NOT NULL,
    aggregate_id VARCHAR(50) NOT NULL,
    event_type VARCHAR(100) NOT NULL,
    payload JSONB NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'NEW',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    published_at TIMESTAMP,
    retry_count INTEGER DEFAULT 0,
    last_attempt TIMESTAMP,
    error_message TEXT
);

CREATE INDEX idx_outbox_status_created ON outbox_messages(status, created_at);
```

Implementation

```java
@Service
@Transactional
public class OrderService {
    public Order createOrder(CreateOrderRequest request) {
        // 1. Save order
        Order order = orderRepository.save(new Order());

        // 2. Save outbox message (same transaction)
        OutboxMessage message = new OutboxMessage();
        message.setEventId(UUID.randomUUID().toString());
        message.setEventType("ORDER_CREATED");
        message.setPayload(toJson(order));
        outboxRepository.save(message);

        return order;
    }
}
```

```java
@Service
public class OutboxProcessor {
    @Scheduled(fixedDelay = 1000)
    public void processOutboxMessages() {
        List<OutboxMessage> messages = outboxRepository.findPendingMessages();

        for (OutboxMessage message : messages) {
            try {
                // Publish to Kafka
                kafkaTemplate.send("order-events", message.getPayload());

                // Mark as processed
                outboxRepository.markAsProcessed(message.getId());
            } catch (Exception e) {
                // Update retry count
                outboxRepository.incrementRetryCount(message.getId(), e.getMessage());
            }
        }
    }
}
```

Hands-On Exercise

1. Create the `outbox_messages` table and an `Order` table in a local Postgres/H2 instance.
2. In one `@Transactional` method, save an order and an outbox row for `ORDER_CREATED` in the same transaction, and confirm (via a forced exception after the order save) that both roll back together.
3. Write a `@Scheduled` poller that reads `status = 'NEW'` rows, logs them (standing in for a real Kafka publish), and marks them `PUBLISHED`.
4. Simulate a publish failure (throw inside the poller for one message) and confirm `retry_count` increments instead of the message being lost.

18. Hibernate vs Spring Data JPA

Explanation

These aren't competing choices — Spring Data JPA is built *on top of* JPA, which Hibernate implements. The value of climbing this stack is less boilerplate: the same "find product by id" operation goes from dozens of lines of raw JDBC to a one-line repository method.

Architecture Layers

```
Application Code
      ↓
Spring Data JPA (Abstraction Layer)
      ↓
JPA (Specification - javax.persistence)
      ↓
Hibernate (Implementation)
      ↓
JDBC (Java Database Connectivity)
      ↓
Database (PostgreSQL, MySQL, etc.)
```

Comparison Table

Feature	JDBC	Hibernate	Spring Data JPA
Abstraction Level	Low	Medium	High
Boilerplate Code	High	Medium	Low
ORM Support	No	Yes	Yes
SQL Generation	Manual	Automatic	Automatic
Caching	No	1st & 2nd Level	Same as Hibernate
Lazy Loading	No	Yes	Yes
Transaction Management	Manual	Declarative	Declarative
Code to CRUD One Entity	~100 lines	~30 lines	~3 lines

Code Comparison

```java
// 1. JDBC - ~50 lines
public Product findById(Long id) {
    // Manual connection, statement, result set
    // Manual mapping
    // Exception handling
    // Resource cleanup
}

// 2. Hibernate - ~5 lines
@Repository
public class ProductHibernateDao {
    public Product findById(Long id) {
        return session.get(Product.class, id);
    }
}
```

```java
// 3. Spring Data JPA - 0 lines!
public interface ProductRepository extends JpaRepository<Product, Long> {
    // All CRUD operations automatically available
    List<Product> findByNameContaining(String name);
}
```

Hands-On Exercise

1. Implement `findById` for a `Product` entity three ways: raw JDBC (`DriverManager` + `PreparedStatement`), a Hibernate `Session.get`, and a Spring Data `JpaRepository`.
2. Time each with a simple stopwatch around 1,000 calls and compare — the point isn't which is "fastest" but to see the boilerplate-vs-abstraction trade-off directly.
3. Turn on `spring.jpa.show-sql=true` and confirm the SQL Spring Data JPA generates matches what you wrote by hand in the JDBC version.

19. Native Query vs JPQL

Explanation

JPQL operates on your entity model (classes and fields), so it's portable across databases; native SQL operates on actual tables and columns, so you lose portability but gain access to database-specific features (window functions, `EXPLAIN` hints, vendor SQL). Criteria API is JPQL built programmatically for type safety and dynamic query construction; derived queries are the simplest but least flexible option.

Comparison

Feature	Native SQL	JPQL	Criteria API	Derived Query
Performance	Best	Good	Moderate	Good
Portability	No: No	Yes: Yes	Yes: Yes	Yes: Yes
Type Safety	No: No	No: No	Yes: Yes	Yes: Yes
Dynamic Queries	Hard	Hard	Yes: Easy	No: No
Database Features	Yes: All	No: Limited	No: Limited	No: No
Learning Curve	Low	Low	High	Lowest
Best For	Complex SQL	Standard Queries	Dynamic Queries	Simple Queries

Examples

```
// 1. Native Query
@Query(value = "SELECT * FROM products WHERE price > :price", nativeQuery = true)
List<Product> findExpensiveProducts(@Param("price") BigDecimal price);

// 2. JPQL
@Query("SELECT p FROM Product p WHERE p.price > :price")
List<Product> findExpensiveProducts(@Param("price") BigDecimal price);

// 3. Criteria API
CriteriaBuilder cb = entityManager.getCriteriaBuilder();
CriteriaQuery<Product> query = cb.createQuery(Product.class);
Root<Product> root = query.from(Product.class);
query.select(root).where(cb.gt(root.get("price"), price));

// 4. Derived Query
List<Product> findByPriceGreaterThan(BigDecimal price);
```

Best Practices

```
// 1. Use Projections for Read-Only Queries
@Query("SELECT new com.example.dto.ProductSummaryDTO(p.id, p.name, p.price) " +
       "FROM Product p WHERE p.price > :price")
List<ProductSummaryDTO> findProductSummaries(@Param("price") BigDecimal price);

// 2. Use JOIN FETCH to Avoid N+1
@Query("SELECT p FROM Product p JOIN FETCH p.category WHERE p.price > :price")
```

```java
List<Product> findProductsWithCategory(@Param("price") BigDecimal price);

// 3. Use Pagination
@Query("SELECT p FROM Product p WHERE p.price > :price")
Page<Product> findProductsWithPagination(@Param("price") BigDecimal price, Pageable pageable);
```

Hands-On Exercise

1. Implement the same "products more expensive than X" query four ways: native SQL, JPQL, Criteria API, and a derived query method.
2. Build a dynamic search endpoint (filter by optional name, min price, max price, category) using Criteria API's `Predicate` list, and confirm it works when only some filters are supplied.
3. Add a projection DTO query (`SELECT new ...`) and confirm via `show-sql` that it selects only the needed columns instead of the full entity.
4. Add `Pageable` support to one query and call it with different page sizes/sort orders.

20. Associations in JPA

Explanation

Getting associations right is mostly about knowing which side “owns” the relationship (the side with the `@JoinColumn`) and which side is just `mappedBy` for convenience. Get this backwards and you’ll either get an extra join table you didn’t want, or updates that silently don’t persist because you set the non-owning side.

Association Types

Association	Annotation	Example
One-to-One	`@OneToOne`	User ↔ Profile
One-to-Many	`@OneToMany`	User → Orders
Many-to-One	`@ManyToOne`	Orders → User
Many-to-Many	`@ManyToMany`	Student ↔ Courses

Examples

```
// 1. One-to-One
@Entity
public class User {
    @Id
    @GeneratedValue
    private Long id;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "profile_id")
    private Profile profile;
}

// 2. One-to-Many / Many-to-One
@Entity
public class User {
    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL)
    private List<Order> orders = new ArrayList<>();
}

@Entity
public class Order {
    @ManyToOne
    @JoinColumn(name = "user_id")
    private User user;
}

// 3. Many-to-Many
@Entity
public class Student {
    @ManyToMany
    @JoinTable(
        name = "student_courses",
        joinColumns = @JoinColumn(name = "student_id"),
```

```
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
    private Set<Course> courses = new HashSet<>();
}
```

Hands-On Exercise

1. Model `User` ↔ `Profile` as `@OneToOne` and confirm inserting a `User` with a `Profile` set cascades the insert to `Profile` as well.
2. Model `User` → `Order` as bidirectional `@OneToMany`/`@ManyToOne`, and confirm that setting `order.setUser(user)` (the owning side) — not `user.getOrders().add(order)` alone — is what actually persists the foreign key.
3. Model `Student` ↔ `Course` as `@ManyToMany` with a join table, and add/remove a course from a student's set to confirm rows appear/disappear from the join table.
4. Deliberately forget to set the owning side of a bidirectional relationship and observe the foreign key column staying `NULL` in the database — this is the single most common JPA association bug.

21. Inheritance in JPA

Explanation

JPA offers three ways to map a Java class hierarchy onto relational tables, and the right choice depends on how different the subclasses are and how important query performance is versus normalized storage. `SINGLE_TABLE` is fastest but wastes space on unused columns; `JOINED` is fully normalized but costs a join per query; `TABLE_PER_CLASS` avoids both but makes polymorphic queries across the whole hierarchy expensive.

Inheritance Strategies

```
// 1. SINGLE_TABLE (Default)
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "employee_type")
public class Employee { /*...*/ }

@DiscriminatorValue("FULLTIME")
public class FullTimeEmployee extends Employee { /*...*/ }

// 2. TABLE_PER_CLASS
@Entity
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public abstract class Vehicle { /*...*/ }

@Entity
@Table(name = "cars")
public class Car extends Vehicle { /*...*/ }

// 3. JOINED
@Entity
@Inheritance(strategy = InheritanceType.JOINED)
public class Animal { /*...*/ }

@Entity
@Table(name = "dogs")
@PrimaryKeyJoinColumn(name = "animal_id")
public class Dog extends Animal { /*...*/ }
```

Comparison

Strategy	Tables	Best For	Pros	Cons
SINGLE_TABLE	1 table	Simple hierarchies	Best performance	NULL columns wasted
TABLE_PER_CLASS	One per class	Simple, no joins	No NULL columns	Polymorphic queries expensive
JOINED	One per class	Complex hierarchies	Normalized	Many joins

Hands-On Exercise

1. Implement `Employee` → `FullTimeEmployee`/`ContractEmployee` under `SINGLE_TABLE` and inspect the generated table — confirm there's a discriminator column and NULL-able fields specific to each subtype.
2. Re-implement the same hierarchy under `JOINED` and compare the schema (separate tables linked by shared primary key) and the SQL generated for a simple `findAll()`.
3. Re-implement under `TABLE_PER_CLASS` and try a polymorphic query (`entityManager.createQuery("FROM Employee")`) — inspect the SQL and note the `UNION` it needs to generate.
4. Write a one-paragraph recommendation for which strategy you'd pick for a hierarchy with 10+ very different subclasses, and justify it using the trade-offs above.

22. Spring Boot Internal Working

Explanation

Understanding the startup sequence demystifies a lot of "why doesn't my bean exist yet" bugs. The short version: Spring first builds and configures the `ApplicationContext` (resolving bean definitions, applying `BeanFactoryPostProcessors`, then instantiating and wiring beans), and only *after* that starts the embedded web server — which is why a `CommandLineRunner` runs before the app is "ready to serve requests" in the truest sense, but after all beans exist.

Startup Process

```
1. main() method invoked
   ↓
2. SpringApplication.run() called
   ↓
3. SpringApplication instance created
   ↓
4. Environment prepared
   ↓
5. Banner printed
   ↓
6. ApplicationContext created
   ↓
7. ApplicationContext prepared
   ↓
8. ApplicationContext refreshed
   ↓
9. Bean instantiation and DI completed
   ↓
10. Embedded server started
   ↓
11. Application ready
```

ApplicationContext Refresh Process

```java
public void refresh() {
    // 1. Prepare context
    prepareRefresh();

    // 2. Obtain fresh bean factory
    ConfigurableListableBeanFactory beanFactory = obtainFreshBeanFactory();

    // 3. Prepare bean factory
    prepareBeanFactory(beanFactory);

    // 4. Post-process bean factory
    postProcessBeanFactory(beanFactory);

    // 5. Invoke BeanFactoryPostProcessors
    invokeBeanFactoryPostProcessors(beanFactory);

    // 6. Register BeanPostProcessors
    registerBeanPostProcessors(beanFactory);
```

```
    // 7. Initialize message source
    initMessageSource();

    // 8. Initialize application event multicaster
    initApplicationEventMulticaster();

    // 9. Initialize special beans
    onRefresh();

    // 10. Register listeners
    registerListeners();

    // 11. Finish bean factory initialization
    finishBeanFactoryInitialization(beanFactory);

    // 12. Finish refresh
    finishRefresh();
}
```

Bean Creation Process

```
public Object createBean(String beanName, RootBeanDefinition mbd, Object[] args) {
    // 1. Resolve bean class
    Class<?> resolvedClass = resolveBeanClass(mbd, beanName);

    // 2. Give BeanPostProcessors a chance to return proxy
    Object bean = resolveBeforeInstantiation(beanName, mbd);
    if (bean != null) {
        return bean;
    }

    // 3. Actual bean creation
    Object beanInstance = doCreateBean(beanName, mbd, args);

    // 4. Register bean as disposable
    registerDisposableBeanIfNecessary(beanName, beanInstance, mbd);

    return beanInstance;
}
```

Hands-On Exercise

1. Add log statements (or breakpoints) inside a custom `BeanFactoryPostProcessor`, a custom `BeanPostProcessor`, a bean constructor, and an `ApplicationListener<ApplicationReadyEvent>` — start the app and record the order they fire in.
2. Confirm that `BeanFactoryPostProcessor` runs before any bean is instantiated, and `BeanPostProcessor` wraps around each individual bean's construction.
3. Add a `CommandLineRunner` and confirm it runs after all beans exist but log the exact moment relative to `ApplicationReadyEvent`.
4. Write a one-sentence explanation, in your own words, of why `@Value` properties aren't available inside a static field or a `@PostConstruct`-free constructor that runs "too early."

23. ACID Properties

Explanation

ACID is the contract a relational database gives you for a single transaction. The one interviewers most often probe on is Isolation, because it's the only one with tunable levels that trade correctness guarantees for performance — the rest (Atomicity, Consistency, Durability) are generally all-or-nothing guarantees the database provides by default.

What is ACID?

```
/*
    Atomicity   → All or nothing
    Consistency → Valid state before → after
    Isolation   → Concurrent transactions don't interfere
    Durability  → Changes survive crashes
*/
```

Examples

```
@Transactional
public void transferMoney(Long fromId, Long toId, BigDecimal amount) {
    // ATOMICITY: All operations succeed or none
    Account from = accountRepository.findById(fromId).orElseThrow();
    Account to = accountRepository.findById(toId).orElseThrow();

    from.setBalance(from.getBalance().subtract(amount));
    to.setBalance(to.getBalance().add(amount));

    accountRepository.save(from);
    accountRepository.save(to);

    // CONSISTENCY: Business rules maintained
    // ISOLATION: Transaction isolation level applies
    // DURABILITY: Changes are persistent on commit
}
```

Hands-On Exercise

1. Implement `transferMoney` and throw an exception right after debiting `from` but before crediting `to` — confirm the debit rolls back too (Atomicity).
2. Add a `@PrePersist`/database `CHECK` constraint that balances can't go negative, and confirm a transfer that would overdraw is rejected (Consistency).
3. Open two database sessions (e.g., two `psql` terminals) and manually run overlapping transactions against the same account to observe how isolation prevents/allows interleaved reads — you'll use this directly in the next section's exercise.
4. Kill the application process mid-way through a committed transaction (after commit, before the process would naturally exit) and confirm on restart the change is still there (Durability).

24. Isolation Levels

Explanation

Each isolation level closes off one more category of concurrency anomaly, at the cost of more locking and lower throughput. `READ_COMMITTED` is the default for most databases (avoids dirty reads only); `REPEATABLE_READ` also prevents a row you've already read from changing under you mid-transaction; `SERIALIZABLE` behaves as if transactions ran one at a time, preventing phantom rows too, but scales the worst.

Isolation Levels

Isolation Level	Dirty Reads	Non-Repeatable Reads	Phantom Reads	Performance
READ_UNCOMMITTED	No: Allowed	No: Allowed	No: Allowed	Best
READ_COMMITTED	Yes: Prevented	No: Allowed	No: Allowed	Good
REPEATABLE_READ	Yes: Prevented	Yes: Prevented	No: Allowed	Moderate
SERIALIZABLE	Yes: Prevented	Yes: Prevented	Yes: Prevented	Worst

Implementation

```
@Transactional(isolation = Isolation.READ_COMMITTED)
public void readCommittedTransaction() {
    // Only reads committed data
    Account account = accountRepository.findById(1L).orElseThrow();
}

@Transactional(isolation = Isolation.REPEATABLE_READ)
public void repeatableReadTransaction() {
    // Consistent reads within transaction
    Account account1 = accountRepository.findById(1L).orElseThrow();
    // Another transaction updates
    Account account2 = accountRepository.findById(1L).orElseThrow();
    // account1 and account2 have same values
}

@Transactional(isolation = Isolation.SERIALIZABLE)
public void serializableTransaction() {
    // Highest isolation - sequential execution
    Account account = accountRepository.findById(1L).orElseThrow();
    account.setBalance(account.getBalance().add(new BigDecimal("100")));
    accountRepository.save(account);
}
```

Hands-On Exercise

1. Using two terminal sessions against the same Postgres database, set one transaction to `READ_UNCOMMITTED`-like behavior (Postgres treats it as `READ_COMMITTED`, so use MySQL if you want a true dirty-read demo) and observe a dirty read of an uncommitted update.
2. Set both sessions to `REPEATABLE_READ`, read the same row twice in one session while the other session commits an update in between, and confirm the first session still sees the original value both times.

3. Set both sessions to `SERIALIZABLE` and force a write-write conflict — confirm one transaction fails with a serialization error rather than silently corrupting data.
4. Map each isolation level to a `@Transactional(isolation = ...)` annotation in a small Spring Boot test and confirm the annotation actually changes behavior against a real database (note: H2's isolation support is limited — Postgres or MySQL is more reliable for this exercise).

Quick Reference Card

Spring Boot Starter Dependencies

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>      <!-- Web MVC -->
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>  <!-- JPA -->
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>  <!-- Security -->
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>      <!-- Testing -->
</dependency>
```

Common Annotations

```
// Spring Boot
@SpringBootApplication
@EnableAutoConfiguration
@ComponentScan

// Spring MVC
@RestController
@RequestMapping
@GetMapping / @PostMapping / @PutMapping / @DeleteMapping
@RequestBody
@PathVariable
@RequestParam
@ModelAttribute

// Spring Data JPA
@Entity
@Table
@Id
@GeneratedValue
@Column
@OneToMany / @ManyToOne / @OneToOne / @ManyToMany
@Transactional

// Spring Security
@PreAuthorize
@Secured

// Caching
@Cacheable
@CachePut
@CacheEvict
```

```
@EnableCaching

// AOP
@Aspect
@Before / @After / @Around / @AfterReturning / @AfterThrowing
@Pointcut
```

Suggested Study Path

For a focused review, work through the hands-on exercises in this order — each one builds a small project you can extend into the next:

1. Sections 1–3 (Fundamentals, Core Spring, MVC) → a single REST controller project.
2. Section 4–5 (JPA, Hibernate) → add persistence to that project with H2.
3. Sections 6, 8 (Security, JWT) → secure the endpoints you built.
4. Sections 10–13 (Exceptions, Transactions, Caching, AOP) → harden the service layer.
5. Sections 9, 14–17 (Microservices, CAP, Saga, CQRS, Outbox) → split the project into two services.
6. Sections 18–21 (JPA deep dive) → revisit persistence with associations and inheritance.
7. Sections 22–24 (Internals, ACID, Isolation) → read-only review, reinforced by the transaction exercises in Section 11.

End of Document.